

Appendix: Free Energy and Active Inference

Daniel Ari Friedman
ORCID: 0000-0001-6232-9096
Email: daniel@activeinference.institute
DOI: 10.5281/zenodo.16887791

October 07, 2026

Contents

1 Appendix: The Free Energy Principle and Active Inference	1
1.1 Overview	1
1.2 Mathematical Formulation and Equation Callouts (Equations linkage)	1
1.3 Four-Fold Partition and Tetrahedral Mapping (Quadrays; Fuller.4D)	2
1.4 How the 4D namespaces relate here	5
1.5 Joint Optimization in the Tetrahedral Framework (Methods linkage)	5
1.6 Implications for AI and Robust Computation	5
1.7 Code, Reproducibility, and Cross-References	5

1 Appendix: The Free Energy Principle and Active Inference

1.1 Overview

The Free Energy Principle (FEP) posits that biological systems maintain their states by minimizing variational free energy, thereby reducing surprise via prediction and model updating. Active Inference extends this by casting action selection as inference under prior preferences. Background: see the concise overview on the [Free energy principle](#) and the monograph [Active Inference \(MIT Press\)](#).

This appendix emphasizes relationships among: (i) the four-fold partition of Active Inference, (ii) Quadrays (Fuller.4D) as a geometric scaffold for mapping this partition, and (iii) information-geometric flows (Einstein.4D analogy) that underpin perception-action updates. For the naming of 4D namespaces used throughout—Coxeter.4D (Euclidean E4), Einstein.4D (Minkowski spacetime analogy), Fuller.4D (Synergetics/Quadrays)—see `02_4d_namespaces.md`.

1.2 Mathematical Formulation and Equation Callouts (Equations linkage)

- Variational free energy (discrete states) — see Eq. (??) in the equations appendix, implemented by `free_energy`: $\mathcal{F} = -\log P(o | s) + \text{KL}[Q(s) \| P(s)]$, where o are observations, s latent states, $Q(s)$ the variational posterior, and $P(s)$ the prior; minimizing $\mathcal{F}$ drives perception.
- Expected free energy (action selection) — see Eq. (??) in the equations appendix, implemented by `expected_free_energy`: $G = \text{KL}[Q(s) \| P(s)] - \mathbb{E}_q[\log P(o | s)] - \log P(o)$, where the epistemic term is the KL divergence between posterior $Q(s)$ and prior $P(s)$ over states (its entropy part, $\mathbb{E}_q[\log Q(s)] = -H[Q(s)]$, is already included, so H is not subtracted again), the ambiguity term $-\mathbb{E}_q[\log P(o | s)]$ penalizes expected surprise about outcomes o , and the pragmatic preference $-\log P(o)$ lowers G for preferred outcomes; agents minimize G when selecting actions.

- Fisher Information Matrix (FIM) as metric — see Eq. (??) in the equations appendix and `fisher_information_matrix`: $F_{ij} = \mathbb{E}[\partial_{\theta_i} \log p(x; \theta) \partial_{\theta_j} \log p(x; \theta)]$, the expected outer product of score functions $\partial_{\theta_i} \log p(x; \theta)$, serving as the Riemannian metric on parameter space θ .
- Natural gradient descent under information geometry — see Eq. (??) in the equations appendix and `natural_gradient_step`: the update $\theta \leftarrow \theta - \eta F^{-1} \nabla_{\theta} L$ preconditions the loss gradient $\nabla_{\theta} L$ by the inverse Fisher metric F^{-1} at step size η ; the implementation solves the damped system $(F + \varepsilon I) \delta = \nabla_{\theta} L$ and moves by $-\eta \delta$. Overview: **Natural gradient**.

Figures: three Active Inference figures follow — the partition tetrahedron, the 4D natural-gradient trajectory (`figure_13_4d_trajectory.png`), and the free-energy landscape (`figure_14_free_energy_landscape.png`).

Discrete variational optimization on the quadray lattice: `discrete_ivm_descent` greedily descends a free-energy-like objective over IVM moves, yielding integer-valued trajectories. See the path animation artifact `discrete_path.mp4` in `quadmath/output/`.

Four-fold partition mapped to Quadray tetrahedron

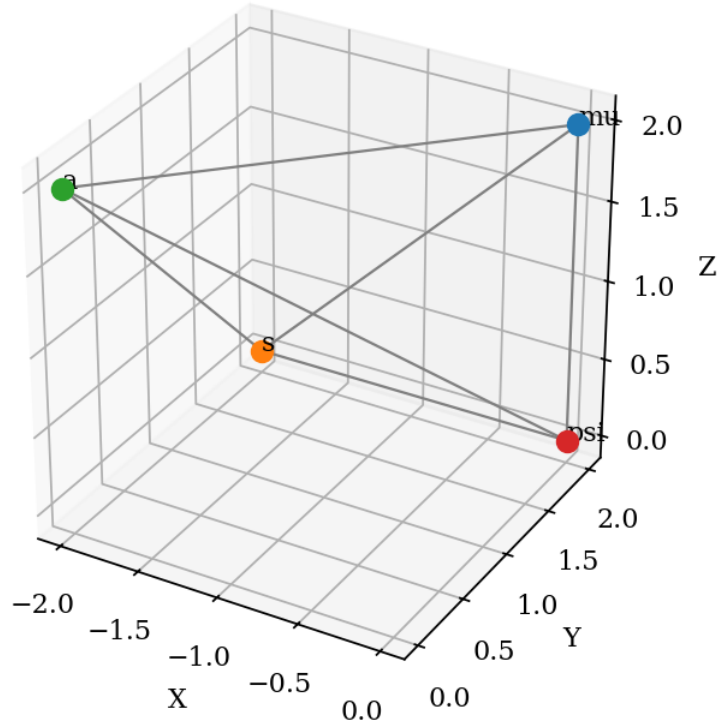


Figure 1: **Active Inference four-fold partition mapped to a Quadray tetrahedron in Fuller.4D**. The four vertices of a regular tetrahedron, embedded in XYZ via the quadray embedding `to_xyz` (axes range -2 to 2), carry the four partition components: μ internal states (blue), s sensory observations (orange), a actions (green), and ψ external causes (red). The gray edges are the six pairwise couplings of the partition — e.g., μ - s perceptual inference and a - ψ control (see the partition section below). Generated by `quadmath/scripts/active_inference_figures.py` (MPLBACKEND=Agg, fixed seed 42).

1.3 Four-Fold Partition and Tetrahedral Mapping (Quadrays; Fuller.4D)

Active Inference partitions the agent-environment system into four coupled states:

- Internal (μ) — agent’s internal states
- Sensory (s) — observations

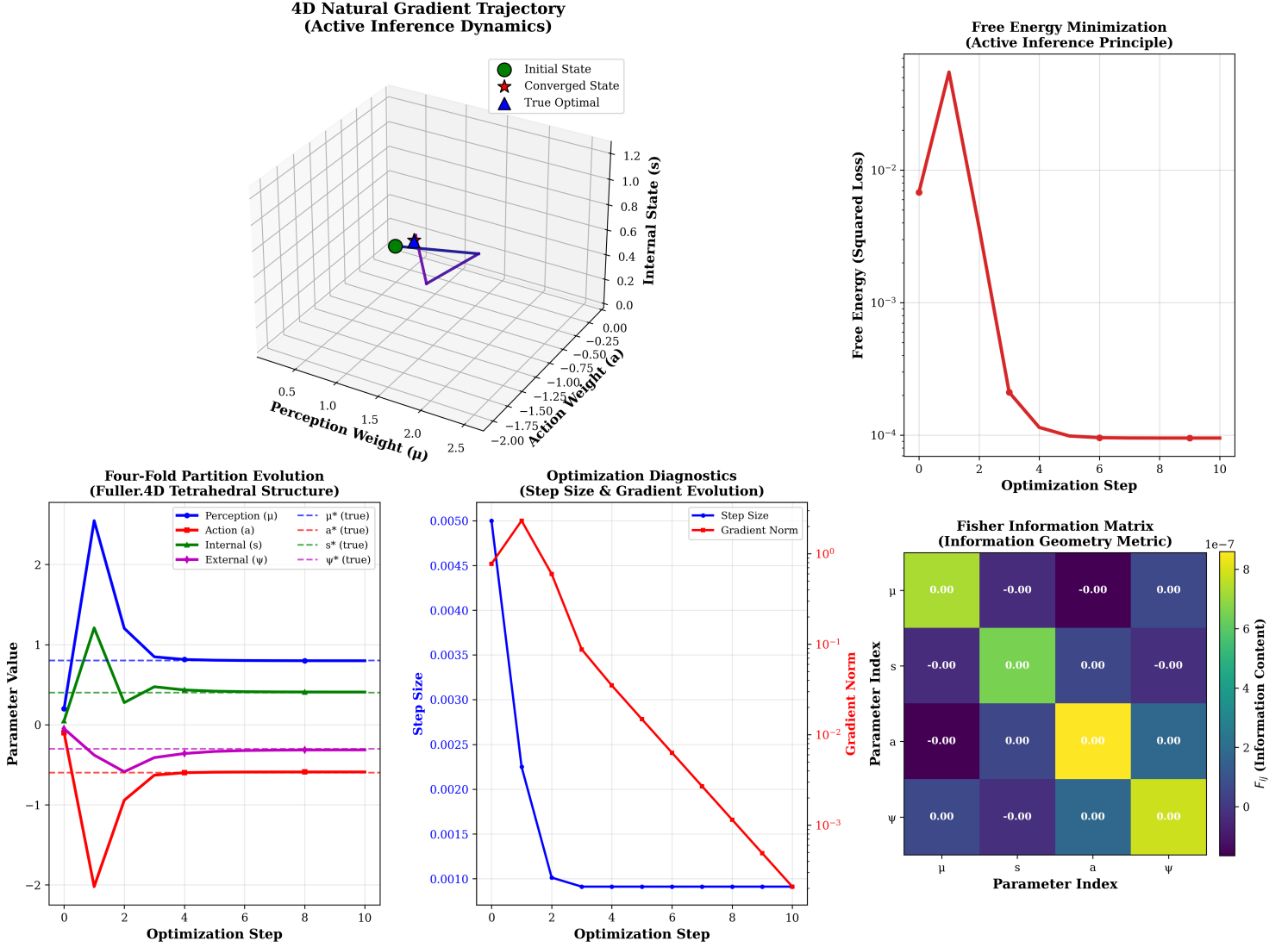


Figure 2: **4D natural-gradient trajectory with Active Inference context** (figure_13_4d_trajectory.png). Top left: the trajectory of natural gradient descent in (μ, a, s) coordinates — perception weight μ , action weight a , internal-state coordinate s — from the initial state (green circle) through the converged state (red star) toward the true optimum (blue triangle), converging in 11 steps with final parameter errors below 0.015. Top right: free energy (squared loss, log scale) falling from about 10^{-1} to 10^{-4} over the 11 optimization steps. Bottom left: evolution of the four partition components μ, a, s, ψ against their true values (dashed lines). Bottom middle: step size (linear scale) and gradient norm (log scale) per step. Bottom right: the 4×4 Fisher information matrix F_{ij} (entries of order 10^{-7}) that preconditions the updates via Eq. (??). Generated by quadmath/scripts/active_inference_figures.py (MPLBACKEND=Agg, fixed seed 42); raw data in figure_13_data.npz.

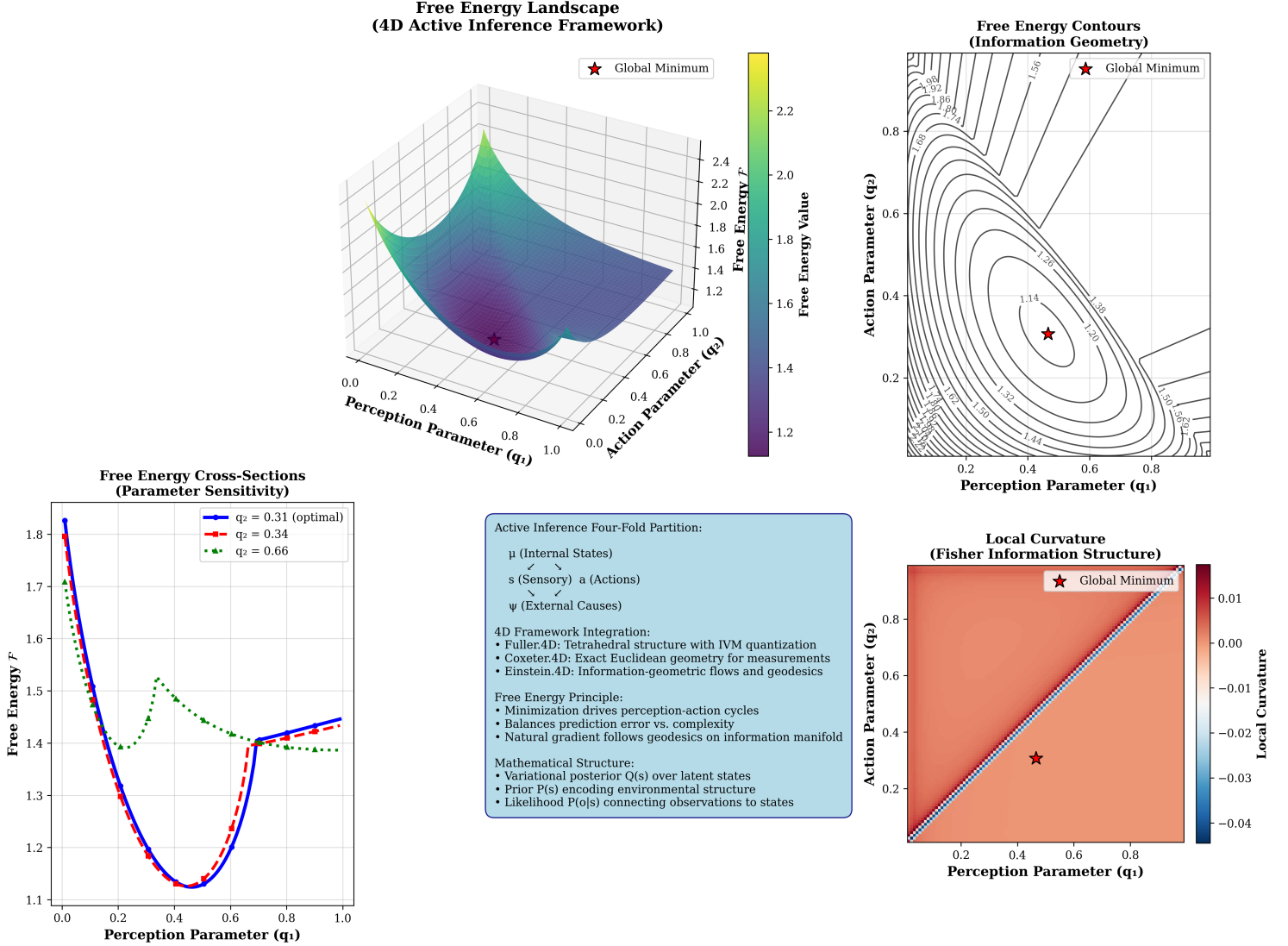


Figure 3: **Free-energy landscape over perception and action parameters** (figure_14_free_energy_landscape.png). Top left: the variational free energy $\mathcal{F}$ of Eq. (??) as a surface over the perception parameter q_1 and action parameter q_2 (values in nats, about 1.2 to 2.4; color bar at right), with the global minimum starred. Top right: level contours of the same surface. Bottom left: one-dimensional cross-sections $\mathcal{F}(q_1)$ at three fixed values of q_2 , showing sensitivity to each parameter. Bottom right: local curvature over (q_1, q_2) (Fisher information structure). The text panel summarizes the four-fold partition (μ, s, a, ψ) , the roles of Fuller.4D/Coxeter.4D/Einstein.4D, and the generative-model objects $Q(s)$, $P(s)$, $P(o | s)$. Generated by quadmath/scripts/active_inference_figures.py (MPLBACKEND=Agg, fixed seed 42); raw data in figure_14_data.npz.

- Active (a) — actions
- External (ψ) — latent environmental causes

See, for an overview of this partition and generative process formulations, the [Active Inference review](#) and the general entry on [Active inference](#).

Tetrahedral mapping via Quadrays (Fuller.4D): assign each state to a vertex of a tetrahedron, using Quadray coordinates (A, B, C, D) with non-negative components and at least one zero after normalization. One canonical mapping is $A \leftrightarrow \mu$ (internal), $B \leftrightarrow s$ (sensory), $C \leftrightarrow a$ (active), $D \leftrightarrow \psi$ (external). The edges capture the pairwise couplings (e.g., μ - s for perceptual inference; a - ψ for control). Integer tetravolume then quantifies the “coupled capacity” region spanned by jointly feasible states in a time slice; see Quadray and tetravolume methods in `03_quadray_methods.md`.

Interpretation note: this Quadray-based mapping is a didactic geometric scaffold. It is not standard in the Active Inference literature, which typically develops the four-state partition in probabilistic graphical terms. Our use highlights structural symmetries and discrete volumetric quantities available in Fuller.4D, building on the computational foundations developed in the [4dsolutions ecosystem](#) for tetrahedral modeling and volume calculations. See the [Resources](#) section for comprehensive details on the computational implementations.

Code linkage (no snippet): see `example_partition_tetra_volume` in `src/quadmath/core/examples.py` and the partition tetrahedron figure above.

1.4 How the 4D namespaces relate here

- Fuller.4D (Quadrays): geometric embedding of the four-state partition on a tetrahedron; integer tetravolumes and IVM moves provide discrete combinatorial structure.
- Coxeter.4D (Euclidean E4): exact Euclidean measurements (e.g., Cayley–Menger determinants) for tetrahedra underlying volumetric comparisons and scale relations.
- Einstein.4D (Minkowski analogy): information-geometric flows (natural gradient, metric-aware updates) supply a continuum picture for perception–action dynamics.

The three roles are complementary: Fuller.4D encodes partition structure, Coxeter.4D provides exact metric geometry for static comparisons, and Einstein.4D guides dynamical descent.

1.5 Joint Optimization in the Tetrahedral Framework (Methods linkage)

- Perception: update μ to minimize prediction error on s under the generative model; `perception_update` integrates the flow $\dot{\mu} = D\mu - \partial_{\mu}F$, where D is the model’s derivative operator and F the free energy of Eq. (??).
- Action: select a that steers ψ toward preferred outcomes; `action_update` descends the action gradient, $a \leftarrow a - \eta \partial_a F$ at step size η .

Continuous-time flows (Einstein.4D analogy for metric/geodesic intuition): see `perception_update` and `action_update` in `src/quadmath/inference/information.py`; their discrete Quadray counterpart is `discrete_ivm_descent` in `src/quadmath/optimize/discrete_variational.py`, described above.

1.6 Implications for AI and Robust Computation

FEP/Active Inference provide algorithms that unify perception and action under uncertainty, offering biologically plausible alternatives to standard RL with adaptive exploration and robust decision-making. See [applications in AI \(arXiv:1907.03876\)](#).

1.7 Code, Reproducibility, and Cross-References

– Equation references: [Eq. \(Free Energy\)](#), [Eq. \(FIM\)](#), [Eq. \(Natural Gradient\)](#) in `08_equations_appendix.md`. – Code anchors (for readers who want to run experiments): `free_energy`, `fisher_information_matrix`, `natural_gradient_step`, `perception_update`, `action_update`, and `discrete_ivm_descent` in `src/quadmath/inference/information.py` and `src/quadmath/optimize/discrete_variational.py`.

Demo and figures generated by `quadmath/scripts/information_demo.py` and `quadmath/scripts/active_inference_figures.py` output to `quadmath/output/`:

- **Active Inference Visualizations (embedded above)**: `figure_13_4d_trajectory.png`, `figure_14_free_energy_landscape.png`, `partition_tetrahedron.png`
- **Information Geometry Visualizations** (rendered in the Optimization in 4D section): `fisher_information_matrix.png`, `fisher_information_eigenspectrum.png`, `natural_gradient_path.png`, `free_energy_curve.png`
- **Raw data**: `figure_13_data.npz`, `figure_14_data.npz`, `fisher_information_matrix.csv`, `fisher_information_matrix.npz` (`F`, `grads`, `X`, `y`, `w_true`, `w_est`), `fisher_information_eigenvalues.csv`, `fisher_information_eigensystem.npz`
- **External validation**: Cross-reference with volume calculations and tetrahedral modeling tools from the [4dsolutions ecosystem](#). See the [Resources](#) section for comprehensive details.